

# Reinforcement Learning With Reward Machines in Stochastic Games

Reinforcement Learning Course - Final Project — 06-2026

---

**Matteo Liotta**

`matteo.liotta@studenti.units.it`

**Mattia Simonutti**

`mattia.simonutti@studenti.units.it`

University of Trieste

*Department of Mathematics and Geosciences*

# Problem Setting

- **Problem:** How to tackle complex tasks in **multi-agent RL** where reward functions are **non-Markovian**?
- **Proposed Approach:** Usage of **Reward Machines** to incorporate the high-level knowledge of the task. By RM state incorporation, form an augmented product stochastic game, **converting non-Markovian rewards** into **standard Markovian ones** [1].
- **Goal:** Learn and converge to the optimal best-response strategy at a **Nash equilibrium** for each agent in a **general-sum game setting**.

## Basic Definitions

---

# Stochastic Games with Non-Markovian Rewards

## Definition — Two-Player General-Sum Stochastic Game (SG)

Modeled as the tuple  $\mathcal{G} = (S, s_I, A_e, A_a, p, R_e, R_a, \gamma)$ , where:

- **Finite State Space ( $S$ ):** consisting of states  $s = [s_e; s_a]$ , with  $s_e, s_a$  substates for *Ego* and *Adv* agents respectively.  $s_I \in S$  is the initial state.
- **Action Spaces:** Each agent has its own finite set of actions  $A_e$  and  $A_a$ .
- **Probabilistic Transition Function:**  $p : S \times A_e \times A_a \times S \rightarrow [0, 1]$
- **Non\*-Markovian Reward functions:**  $R_e \text{ or } a : (S \times A_e \times A_a)^+ \times S \rightarrow \mathbb{R}$  map from **complete trajectory histories** to real value domains, specifying the n-m rewards for both agents.

(\*) Usual definition assumes system Markovianity.

# Trajectories and Rewards

- **Trajectory:** Sequence of states and actions  $t_{0:k+1} = s_0 a_{\epsilon,0} a_{\alpha,0} s_1 \dots s_k a_{\epsilon,k} a_{\alpha,k} s_{k+1}$  with  $s_0 = s_I$ .
- **Reward Sequence:** Given a **trajectory**, each agent ( $i \in \{\epsilon, \alpha\}$ ) receives reward sequence  $r_{i,1} \dots r_{i,k}$ , where  $r_{i,k} = R(s_0, a_{\epsilon,0}, a_{\alpha,0}, \dots, s_k)$ .
- **Shared Observation:** Both agents **observe the same** trajectory.
- **Discounted Cumulative Reward:** Given a **trajectory**  $t_{0:k+1}$ , agents achieve a DCR reward  $\sum_{t=0}^k \gamma^t r_{i,t}$
- **Finite episodes assumption:** The game is episodic with a finite horizon, ensuring that the cumulative reward is well-defined.

# Strategies and Objectives

- **Markovian Strategy:** Agents are equipped with a  $\pi_i : S \times A_i \rightarrow [0, 1]$   $i \in \{\epsilon, \alpha\}$  strategy, defining action given state selection probability.
- **Objective:** Maximize the **expected** discounted cumulative reward given policy of both agents, from both point of view.

## Definition — SG Expected Discounted Cumulative Reward

Given agent  $i \in \{\epsilon, \alpha\}$ , the expected DCR under policies  $\pi_\epsilon$  and  $\pi_\alpha$  is defined as:

$$\tilde{v}_i(s, \pi_\epsilon, \pi_\alpha) = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}(r_{i,t} | \pi_\epsilon, \pi_\alpha, s_0 = s)$$

# Nash Equilibrium

Each agent's strategy is a **best-response to the other agent's strategy**, if each agent **cannot improve its own reward** by changing its strategy, given a **fixed opponent's policy**.

## Definition — Nash Equilibrium in Stochastic Games

For a stochastic game  $\mathcal{G} = (S, s_I, A_e, A_a, R_e, R_a, \gamma, p)$ , the strategies of the ego agent,  $\pi_e^*$ , and the adversarial agent,  $\pi_a^*$ , are at a **Nash equilibrium** if:

$$\tilde{v}_e(s, \pi_e^*, \pi_a^*) \geq \tilde{v}_e(s, \pi_e, \pi_a^*)$$

$$\tilde{v}_a(s, \pi_e^*, \pi_a^*) \geq \tilde{v}_a(s, \pi_e^*, \pi_a)$$

hold for any  $\pi_e$  and  $\pi_a$ .

## Objectives (cont'd)

The **actual objective** of each agent is to find the policy  $\pi_i^*$ , maximising the **expected discounted cumulative reward** considering the other agent's action once it reaches the game's Nash equilibrium

### Agent Objectives at Nash Equilibrium

$$\pi_{\epsilon}^* = \arg \max_{\pi_{\epsilon}} \tilde{v}_i(s, \pi_{\epsilon}, \pi_{\alpha}^*)$$

$$\pi_{\alpha}^* = \arg \max_{\pi_{\alpha}} \tilde{v}_i(s, \pi_{\epsilon}^*, \pi_{\alpha})$$



## Definition — Reward Machine (RM)

Finite state automaton designed to encode non-Markovian rewards with state transitions and output functions:  $\mathcal{A} = (V, v_I, 2^{\mathcal{P}}, \mathbb{R}, \delta, \sigma)$

- **Input Alphabet ( $2^{\mathcal{P}}$ ):** Interprets propositional logic variables  $\mathcal{P}$  generated from lower-level environment updates.
- **Set of RM states ( $V$ ):** Finite collection of internal tracking states, with  $v_I$  as the initial state.
- **Transition function:** Deterministic state progression  $\delta : V \times 2^{\mathcal{P}} \rightarrow V$ .
- **Output function:** Discrete scalars along active trajectory  $\sigma : V \times 2^{\mathcal{P}} \rightarrow \mathbb{R}$ .

# Reward Machines example visualization

We define Reward Machines as a formalism to express **task specification**, decomposed into **several stages**.

## Example — variant of the PAC-MAN game

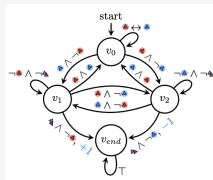
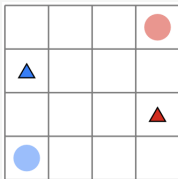


Figure 1.,2. Variant of the PAC-MAN game is a symmetric zero-sum game where two agents try to capture each other. [1]

# Connecting SG and RMs

A link between RM and SG is required

## Labelling Function

$$L : S \times A_e \times A_a \times S \rightarrow 2^{\mathcal{P}}$$

$l_t = L(s_t, a_{a,t}, a_{e,t}, s_{t+1})$  is the set of **true propositions** in  $\mathcal{P}$  at step  $t$ .

## RM can encode non-Markovian RF of a stochastic game

Given a SG, the RM  $\mathcal{A}_i$  **encodes** the non-markovian reward function  $R_i$  of the game, meaning that **for each trajectory**  $s_0 a_{e,0} a_{a,0} s_1 \dots s_k a_{e,k} a_{a,k} s_{k+1}$  (i.e.  $l_0 l_1 \dots, l_k$ ) it outputs the **reward sequence**  $r_{i,0} r_{i,1} \dots, r_{i,k}$ .

I.e. the RM maps **any sequence of events to a sequence of rewards**.

## Q-learning with RM for SG

---

# Problem formulation

The paper focuses on **two-agent general-sum stochastic games** (SG) with NM reward functions, where each agent is given a (different) task to complete.

Task completion **depends on the other's agent behavior**.

- Each agent as a separate reward machine.
- Agents operate in a shared and adversarial environment.

Then, the SG with non-markovian reward functions is formalized as **a product stochastic game** to obtain **Markovian rewards** using **reward machines**.

# Markovian formulation: SGRM

## Definition — Stochastic Game with Reward Machines (SGRM)

Given an SG  $\mathcal{G}$  and RMs  $\mathcal{A}_\epsilon, \mathcal{A}_\alpha$ , a SGRM is defined as a product stochastic game:

$$\mathcal{H} = (S', s'_I, A_\epsilon, A_\alpha, p', R'_\epsilon, R'_\alpha, \gamma)$$

- **Augmented State Space:**  $S' = S \times V_\epsilon \times V_\alpha$ . An **augmented state** is  $s' = (s, v_\epsilon, v_\alpha)$ .
- **Transition Probabilities:**  $p'(s', a_\epsilon, a_\alpha, \bar{s}') = p(s, a_\epsilon, a_\alpha, \bar{s})$  if  $\bar{v}_i = \delta_i(v_i, L(s, a_\epsilon, a_\alpha, \bar{s}))$  for both  $i \in \{\epsilon, \alpha\}$ , and 0 otherwise.
- **Markovian Reward Functions:**  $R'_i(s', a_\epsilon, a_\alpha, \bar{s}') = \sigma_i(v_i, L(s, a_\epsilon, a_\alpha, \bar{s}))$ ,  $\forall i$

The **SGRM** is proved to have a Markovian **RF** to express the original **non-markovian RF** of the **SG**.

# SGRM Steps

The agent consider  $S'$  to select actions: at **each time step simultaneous action selection and execution** occurs; this causes the **game state transition** from  $s$  to  $s'$ .

The SGRM labelling function  $L$  returns the **high-level event associated**,  $l_t$ . Using current label, the RM performs **RM state transition**, by moving from  $(v_e, v_a)$  to  $(v'_e, v'_a)$ , using each agent's specific RM, i.e.  $v'_i = \delta(v_i, L(s, a_e, a_a, s'))$

The **transition of both game states and RM states** leads to the step **reward** for the agent  $r_{i,t} = \sigma(v_i, L(s, a_e, a_a, s'))$

# Nash Equilibrium and Q-functions in SGRM

Q-functions are used at Nash equilibrium.

## QSG-RM

Q-function at a Nash equilibrium is the EDCR obtained by agent  $i$  when **both agents** follow a **joint** Nash Equilibrium strategy from the next period on.

So, it depends on **both agents actions**, and it's defined over the augmented state space.

## Definition — Q-Function at a Nash Equilibrium in SGRM

Given agent  $i$ , given augmented state space  $S \times V_e \times V_a$ , we define

$$q_i^*(s, v_e, v_a, a_e, a_a) = r_i + \gamma \sum_{s', v'} p(s', v' | s, v, a_e, a_a) \tilde{v}_i(s', v'_e, v'_a, \pi_e^*, \pi_a^*)$$



## Nash Equilibrium and Q-functions in SGRM (cont'd)

Using the q-function in the augmented space, best response policies are then originally extracted using the **Lemke-Howson method**.

By definition an agent strategy is optimal if the other agent behavior is considered: this requires both agents to **maintain two Q-functions**.

### Agent learned (two) Q-functions

$q_{i,j}$  represents the Q-function for agent  $i$  as learned by agent  $j$ . It represents the **estimation that agent  $j$  does of agent  $i$ 's expected discounted cumulative reward**, assuming a game-theoretic baseline where each plays **accordingly to Nash Equilibrium** (i.e. choosing the best response to the other player -estimated-strategy.)

As a consequence, **each agent can estimate** (its own and) **other player's strategy**  $\pi_{i,j}$ , using the q-function estimation of its EDCR.

# The Stage Game Matrix

## Definition: Stage Game

A two-player stage game is defined as  $(r_\epsilon, r_\alpha)$ , where  $r_i$  is agent  $i$ 's reward function  $R_i$ , over the space of joint actions:

$$r_i = \{R_i(a_\alpha, a_\epsilon) | a_\alpha \in A_\alpha, a_\epsilon \in A_\epsilon\}$$

- In QRM-SG, a stage game is formulated at each time step based on current Q-functions in augmented state space.
- The **Lemke-Howson method** is used to derive best-response strategies at a Nash equilibrium.

# Unified Framework

---

# QRM-SG Algorithm Architecture

## Actor Mechanics (Action Selection)

- Balances policy exploration using an  $\epsilon$ -greedy wrapper around calculated stage-game equilibrium solutions.
- With probability  $1 - \epsilon$ , the model tracks the maximum probability mixed-strategy weights output by the **Lemke-Howson algorithm**.

## Critic Updates (Learning Loop)

Values are updated according to standard stochastic approximation:

$$q_{ji} \leftarrow (1 - \alpha_t)q_{ji} + \alpha_t(r_{i,t} + \gamma \bar{q}_{ji}(s', v'_e, v'_a))$$

where  $\bar{q}_{ji}$  is the expected value scalar at the next augmented state under best-response actions.

# QRM-SG Phase 1: Initialization

## Process Breakdown (Lines 1–4)

**Initialize**  $\alpha_t, \gamma, \epsilon$

$s \leftarrow \text{InitialState}(); v_{\epsilon} \leftarrow v_{I,\epsilon}; v_{\alpha} \leftarrow v_{I,\alpha}$

$q_{\epsilon i}(\cdot), q_{\alpha i}(\cdot) \leftarrow \text{InitialQfunctions}()$

## Analysis:

- Each agent starts with a "dual-view" Q-table, estimating values for self and opponent to allow local equilibrium computation.
- State is defined in the product space  $S \times V_{\epsilon} \times V_{\alpha}$ , ensuring the Markov property is preserved.

## QRM-SG Phase 2: Nash-Action Selection

### Process Breakdown (Lines 7–13)

```
if  $\bar{\epsilon} < \epsilon$  then
     $a_i \leftarrow \text{ChooseActionRandomly}()$ 
else
     $\pi_{\epsilon i}, \pi_{\alpha i} \leftarrow \text{CalculateNashEquilibrium}(q_{\epsilon i}, q_{\alpha i})$ 
     $a_i \leftarrow \operatorname{argmax} \pi_{ij}(a|s, v_{\epsilon}, v_{\alpha})$ 
end if
```

### Analysis:

- **Exploration:** Standard  $\epsilon$ -greedy mechanism balances discovery and optimization.
- **Exploitation:** Agents solve a bimatrix stage game using the **Lemke-Howson method** to find mixed strategy equilibria.

## QRM-SG Phase 3: Reward Machine Dynamics

### Process Breakdown (Lines 14–18)

$$s' \leftarrow \text{ExecuteAction}(s, a_e, a_a)$$
$$l_t \leftarrow L(s, a_e, a_a, s')$$
$$v'_i \leftarrow \delta_i(v_i, l_t); r_{i,t} \leftarrow \sigma_i(v_i, l_t)$$

### Analysis:

- Bridges low-level physical dynamics ( $s \rightarrow s'$ ) with high-level automata state tracking.
- The labeling function  $L$  detects high-level environmental events (e.g., "power base reached").
- Rewards  $r_{i,t}$  are generated by the RM transition, making them context and history-aware.

## QRM-SG Phase 4: Q-Value Update (Critic)

### Process Breakdown (Lines 19–24)

$$\pi_{ji}(\cdot | s', v') \leftarrow \text{Nash}(q_{ei}(s', v'), q_{ai}(s', v'))$$

$$\bar{q}_{ji} \leftarrow \pi_{ei} \pi_{ai} q_{ji}(s', v')$$

$$q_{ji} \leftarrow (1 - \alpha_t) q_{ji} + \alpha_t (r_{j,t} + \gamma \bar{q}_{ji})$$

### Analysis:

- **Bootstrapping:** Updates current Q-values by computing the Nash equilibrium of the future state  $(s', v')$ .
- **Scalarization:**  $\bar{q}_{ji}$  is the expected value of agent  $j$  when both follow the Nash strategy profile.
- **Coupled Update:** Agents must learn both Q-functions concurrently to maintain equilibrium local estimates.



# Theoretical Foundations

---

# Convergence Assumptions

## Assumption 1: Exploration

Every state  $s \in S$ , RM states  $v_e \in V_e$ ,  $v_a \in V_a$ , and actions  $a_e \in A_e$ ,  $a_a \in A_a$ , are visited infinitely often when the number of episodes goes to infinity.

## Assumption 2: Learning Rate

The learning rate  $\alpha_t$  satisfies the following conditions for all  $t$ :

1.  $0 < \alpha_t < 1$ ,  $\sum_t \alpha_t(s, v_e, v_a, a_e, a_a) = \infty$ ,  $\sum_t [\alpha_t(s, v_e, v_a, a_e, a_a)]^2 < \infty$ , and the latter two hold uniformly and with probability 1.
2.  $\alpha_t(s, v_e, v_a, a_e, a_a) = 0$  if  $(s, v_e, v_a, a_e, a_a) \neq (s^t, v_e^t, v_a^t, a_e^t, a_a^t)$ .

## Assumption 3: Stage Game Structure

One of the following conditions holds during learning:

1. Every stage game  $(q_{ei}(s, v_e, v_a), q_{ai}(s, v_e, v_a))$ ,  $i \in \{e, a\}$ , for all  $t, s, v_e$ , and  $v_a$ , has a **global optimal point**  $(\tilde{\pi}_{ei}, \tilde{\pi}_{ai})$  (i.e.,  $\tilde{\pi}_{ji}q_{ji} \geq \tilde{\pi}'_{ji}q_{ji}$  for any  $\tilde{\pi}'_{ji}$ ,  $j \in \{e, a\}$ ) and agents' rewards in this equilibrium are used to update their Q-functions.
2. Every stage game  $(q_{ei}(s, v_e, v_a), q_{ai}(s, v_e, v_a))$ ,  $i \in \{e, a\}$ , for all  $t, s, v_e$ , and  $v_a$ , has a **saddle point**  $(\tilde{\pi}_{ei}, \tilde{\pi}_{ai})$  (i.e.,  $\tilde{\pi}_{ji}\tilde{\pi}_{-ji}q_{ji} \geq \tilde{\pi}'_{ji}\tilde{\pi}_{-ji}q_{ji}$  for any  $\tilde{\pi}'_{ji}$ ,  $\tilde{\pi}_{ji}\tilde{\pi}_{-ji}q_{ji} \leq \tilde{\pi}_{ji}\tilde{\pi}'_{-ji}q_{ji}$  for any  $\tilde{\pi}'_{-ji}$ ,  $j \in \{e, a\}$ ), and agents' rewards in this equilibrium are used to update their Q-functions.

# Theorem 1: Convergence Guarantee

## Theorem 1

Under Assumptions 1, 2 and 3, the sequence  $q_{it} = (q_{\epsilon i}^t, q_{\alpha i}^t)$  at time  $t$ ,  $i \in \{\epsilon, \alpha\}$  updated by:

$$q_{ji}(s, v, a) = (1 - \alpha_t)q_{ji}(s, v, a) + \\ \alpha_t(r_{j,t} + \gamma \pi_{\epsilon i}(\cdot | s', v') \pi_{\alpha i}(\cdot | s', v') q_{ji}(s', v'))$$

for  $j \in \{\epsilon, \alpha\}$ , where  $(\pi_{\epsilon i}(\cdot | s', v'), \pi_{\alpha i}(\cdot | s', v'))$  is the appropriate type of Nash equilibrium solution for the stage game  $(q_{\epsilon i}^t(s', v'), q_{\alpha i}^t(s', v'))$ , converges to the Q-functions at a Nash equilibrium  $(q_{\epsilon i}^*, q_{\alpha i}^*)$ .

## Algorithm 1: QRM-SG Full Pseudocode

```
1: Initialize  $q_{ei}$  and  $q_{ai}$  for all  $i \in \{e, a\}$ 
2: for episode  $l = 1, 2, \dots$  do
3:    $s \leftarrow s_l, v_e \leftarrow v_{l,e}, v_a \leftarrow v_{l,a}$ 
4:   for  $t = 0$  to  $eplength$  do
5:     Choose action  $a_i$  using  $\epsilon$ -greedy policy and Lemke-Howson
6:     Execute actions, observe  $s'$ , and high-level event  $l_t = L(s, a_e, a_a, s')$ 
7:     Update RM states:  $v'_i \leftarrow \delta_i(v_i, l_t)$  and reward  $r_{i,t} \leftarrow \sigma_i(v_i, l_t)$ 
8:     Compute Nash strategy  $\pi_{ji}(\cdot | s', v')$  at next augmented state
9:     Update Q-functions:
10:     $q_{ji} \leftarrow (1 - \alpha_t)q_{ji} + \alpha_t(r_{j,t} + \gamma \sum \pi \pi q'_{ji})$ 
11:     $s \leftarrow s', v_e \leftarrow v'_{e'}, v_a \leftarrow v'_{a'}$ 
12:   end for
13: end for
```

# Algorithm Analysis and Convergence Properties

- **Off-Policy Learning:** RMs enable decomposition of complex RL problems into structured subproblems for efficient learning.
- **Coupled Learning:** The learning processes of agents are coupled because the high-level events depend on joint actions.
- **Decentralized View:** Each agent maintains a "centralized" perspective by learning everyone's Q-functions to compute equilibria locally.

# Mathematical Foundation: Convergence Proof

The proof relies on viewing the update as a stochastic approximation of a mapping  $F^t$ :

## Key Lemmas from Supplementary Materials

- **Lemma 2:** Establishes general convergence for iterations  $q^{t+1} = (1 - \alpha_t)q^t + \alpha_t[F^t q^t]$ .
- **Lemma 4:** Shows that Q-functions at a Nash equilibrium ( $q^*$ ) are fixed points of  $F^t$ , i.e.,  $q^* = \mathbb{E}[F^t q^*]$ .
- **Lemma 7:** Proves  $F^t$  is a **contraction mapping** under Assumption 3:

$$\|F^t q - F^t \dot{q}\| \leq \gamma \|q - \dot{q}\|$$

## Results and Applications

---



- **Testing Environment:** Evaluated on  $6 \times 6$  (and  $12 \times 12$ ) competitive, sparse-reward grid world variations of Pac-Man.
- **Comparative Baselines:**
  - Nash-Q: Pure spatial state-action tracking.
  - Nash-QAS: Incorporates an unstructured history status vector.
  - MADDPG-SG & MADDPG-AS: Deep multi-agent deterministic policy gradient variants.

# Case Study Empirical Analysis

- **Case Study I (Sequential Ordering):** QRM-SG converges to equilibrium by  $\approx 7,500$  episodes, whereas unaugmented baselines fail entirely.
- **Case Study II & III (Energy Loops):** Explores complex recharge loops with randomized origin points. QRM-SG reaches stability within  $\approx 1,000$  to  $1,500$  runs.
- **Scalability:** Successfully finds the Nash equilibrium on an expanded  $12 \times 12$  map space by episode 21,000.

## Conclusion

---

## Summary & Future Outlook

- **Core Contribution:** Bridges reward machines with MARL to model non-Markovian constraints in stochastic games.
- **Robust Architecture:** Augmented state transformations allow tabular actors to navigate complex environments successfully.
- **Future Directions:** Dynamically learning unknown reward machine configurations simultaneously during policy calculation; extensions to general-sum games with more agents.

## References

---

- [1] Jueming Hu, Jean-Raphaël Gaglione, Yanze Wang, Zhe Xu, Ufuk Topcu, and Yongming Liu.

**Reinforcement learning with reward machines in stochastic games.**

*arXiv preprint arXiv:2305.17372v3*, cs.MA, 2023.